Contratos pelo bot, revisão pela tela e n8n pelas rotas · status de entrega

> **Sem número de bloco.** Trilha da automação, lista **"OCR Contratos (WhatsApp) — Breno"** do ClickUp, numeração própria de 1 a 8. Ver `docs/README.md` e o `CLAUDE.md`.

Data: 2026-09-25 · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`) e n8n Cloud · Plano: `docs/automacao/migracao-telegram-integracao-automacao.md`, decisões 23, 25 e 26.

Resumo

Os três itens pedidos pelo Breno em 25/09 estão **implementados, com validação pendente** — pela regra do `CLAUDE.md`, `npm run validar` só roda sob pedido ou no fechamento.

1. **Contratos pelo bot (decisão 23):** regra em `src/lib/ingestao.ts` e rota `POST /api/ingestao/contrato`. **Só funciona em produção depois do merge:** a rota ainda não está no deploy do `gc-sistema-nine`. Até lá, contrato que chegar vai para revisão com "a entrada automática de contratos ainda não está publicada".
2. **Revisão pela tela:** Reprocessar, Vincular e Descartar no detalhe de `/documentos`.
3. **n8n pelas rotas (decisão 26):** o `Processar Documento` grava proposta e contrato chamando as rotas do sistema; a regra empacotada saiu do n8n.

Nada disso rodou de verdade: o **n8n segue no limite de execuções**.

---

1. Contratos pelo bot

As três relações que o Breno confirmou em 23/09:

RELAÇÃO	COMO
Obra	**lida do PDF** (decisão 27, seção 6) — o contato só autoriza o envio
Proposta citada	a rota procura pelo número **na mesma obra** e grava `proposta_origem_id` (a FK composta exige a mesma obra). Não achou → entra sem vínculo e o grupo é avisado. Valor divergente → só aviso (decisão 16)
Itens	os do PDF, pela mesma regra da proposta (decisões 10 e 11)

- **Por que não a função da sprint 6:** `gerar_contrato_de_proposta` exige proposta **aprovada** e copia os itens *da proposta*. O contrato do bot traz os próprios itens e a proposta citada pode estar em rascunho. O caminho é o do contrato avulso (6.3): nasce **ativo**, histórico de origem (`ativo → ativo`, `por` = uuid do profile de serviço).

- **`src/lib/ingestao.ts`**: a validação de cabeçalho e de itens foi separada (`validarCabecalho`, `montarItens`) e agora serve os dois; `montarIngestaoContrato` é novo. Os **21 testes da proposta** continuaram passando depois da separação.
- **Rota:** mesmo desenho da de proposta — token em tempo constante, documento × empresa, obra × empresa, idempotência por `contrato_criado_id` (200), número repetido → 409 (`mensagemDeErroContrato`), itens recusados → o contrato é apagado.
- **No n8n:** `Analisar e decidir` deixou de mandar contrato para revisão; contrato com valor, obra e número vai para a rota de contrato.

2. Revisão pela tela (`/documentos/[id]`)

Painel **Resolver** para admin e comercial, enquanto o documento está em Processando, Faltam dados ou Precisa de revisão:

AÇÃO	O QUE FAZ
Reprocessar	volta a PENDENTE, gera URL nova do PDF e chama o webhook do n8n (mesma leitura). Pode trocar a obra — o caso "não conseguimos confirmar a que obra"
Vincular	a equipe fez à mão: liga o documento a uma proposta ou contrato da obra → APROVADO
Descartar	PDF errado, duplicado, de outra empresa: status novo DESCARTADO, com o motivo

- **Migration `20260925100000_documentos_revisao.sql`, aplicada em gc-dev** (decisão 25): `DESCARTADO` no CHECK de status, `revisado_por` e `revisado_em`. Tipos regerados: só as colunas novas e a FK.
- As três actions repetem a checagem de perfil e recusam documento já resolvido.
- **Reprocessar não apaga a origem:** o `Criar registro` do n8n passou a não sobrescrever `canal` / `canal_chat_id` quando reaproveita um documento pelo id.

3. n8n pelas rotas (decisão 26)

- **Saíram 8 nós** (`Montar ingestão` com a regra empacotada, `Criar proposta`, `Tem itens?`, `Criar itens`, desfazer, IF de proposta). **Entraram 3:** `Registrar no sistema?`, `Montar pedido da rota` e `Rota - Gravar no sistema` (HTTP com a Credential `5Kf5wOUxCNenfQkd`, 2 tentativas — a rota é idempotente pelo documento).
- `Registrar falha` lê a resposta da rota: **409** → revisão ("já existe..."), **422** → revisão com o motivo da rota, **404** (rota de contrato ainda não publicada) → revisão sem alarme técnico, **outros** → falha técnica.
- O PDF vira anexo da **proposta ou do contrato** (`{empresa}/contratos/{id}/…`).
- `scripts/empacotar-ingestao-n8n.mjs` removido do working tree (estava no commit da sprint 7).

4. Números reais

VERIFICAÇÃO	RESULTADO
`node --test src/lib/ingestao.test.ts`	**25/25** (21 da proposta + 4 do contrato)
`node --test src/lib/documentos.test.ts`	**14/14**
`tsc --noEmit`	limpo nos arquivos desta rodada (o repo teve, por um momento, erro num arquivo da sprint 8, já corrigido por ela)
`next lint`	sem avisos
n8n, offline, nós reescritos	**16/16** — contrato entra, corpo das duas rotas, 409/422/404/500, avisos de proposta e contrato, anexo do contrato
`n8n_validate_workflow`	**0 erros**, 34 nós, 3 gatilhos
Referências a nós inexistentes	nenhuma

5. O que NÃO foi validado — pendências nominais

1. **`npm run validar` não rodou.** Plano estendido: `rotas-esperadas.txt` (42 → 43, `/api/ingestao/contrato`, à mão); uma checagem de dados (colunas de revisão e opções de vínculo); **22 passos de escrita** — rota de contrato (401, criação com proposta citada, ativo/autor/histórico, obra e data, itens com inferência e `UN` → `QTD`, valor = soma, documento vinculado, repetição 200, número repetido 409, proposta inexistente sem vínculo) e revisão (visualizador recusado, motivo curto, reprocessar sem obra, vincular, documento já resolvido, descartar, documento alheio), com limpeza; e o **passo 30 do navegador** (painel Resolver e validação do descarte). O reprocessar que chama o n8n de verdade não é exercitado.
2. **A rota de contrato não está publicada.** Precisa de commit, PR e merge; e o sync do fork (a Action `.github/workflows/sincronizar-fork.yml` também só funciona depois do merge).
3. **Nada rodou no n8n** (limite de execuções): nem proposta nem contrato pelas rotas.
4. **As ações de revisão não foram abertas na tela.**
5. **A rota de contrato fica fora da camada runtime**, pelo mesmo motivo da de proposta (401 em vez de 307).

---

6. A obra sai do documento (decisão 27) — 25/09

Correção do Breno sobre o desenho acima: *o contato é só quem pode enviar; a obra o bot tira do PDF.*

- **`src/lib/obra-do-documento.ts`** (novo, **10 testes**): `identificarObra` casa as pistas lidas com as obras da empresa, nesta ordem — **código** (igual, ou o código cadastrado aparece no texto lido, ex.

`Obra: EB-25-08-0044 - ESTAÇÃO FASHION`), **nome** (igual, ou um contém o outro, mínimo 6 letras), **cliente** (só se o cliente tem uma obra). Acento, caixa e pontuação não contam. Aceita só resultado **único**; ambíguo ou nada → revisão com as candidatas.

- **As duas rotas**: sem `obraId`, consultam as obras da empresa e chamam `identificarObra`. Não achou → **422 "Obra não identificada: ..."** → o documento vai para revisão, e a pessoa resolve com **Reprocessar escolhendo a obra**. Achou → a resposta diz como (`obraIdentificadaComo`).
- **n8n**: o prompt pede `obra_codigo` e `obra_nome`; a leitura não manda mais para revisão por falta de obra; o pedido à rota leva as pistas; o bot não usa mais a obra do contato; o aviso ao grupo diz "obra identificada pelo código/nome/cliente no documento". Testado offline **6/6**, publicado, `n8n_validate_workflow` 0 erros.
- **Contatos**: o formulário ficou com código e nome; a tabela mostra "cadastrado em" no lugar da obra. **Envio pela tela**: obra opcional ("Identificar pelo documento").
- **Plano de validação**: os passos de contato e de envio que exigiam obra foram corrigidos (o "envio sem obra é recusado" saiu — ele passaria a chamar o n8n de verdade); 2 passos novos na rota de contrato: obra achada pelo código do documento, e obra inexistente → 422.

Atenção até o merge: a rota de proposta **publicada hoje** no `gc-sistema-nine` é a antiga, que exige `obraId`. Com o n8n já mandando sem obra, toda proposta que chegar antes do merge vai para revisão com "Campos obrigatórios ausentes: obraId". O merge resolve as duas coisas (esta e a rota de contrato). Hoje o n8n está no limite de execuções, então nada chega.

VERIFICAÇÃO	RESULTADO
`node --test src/lib/obra-do-documento.test.ts`	**10/10**
`node --test src/lib/contatos.test.ts`	**11/11**
`node --test src/lib/ingestao.test.ts`	**25/25**
`tsc --noEmit` / `next lint`	limpos
n8n offline	**6/6**

7. Um contrato por obra e a conferência "aceitar ou não" — 25/09

7.1 Uma obra, um contrato (decisão 28)

A rota de contrato confere, antes de criar, se a obra já tem contrato que não esteja rescindido; se tem, **409** "A obra já tem um contrato vigente (número)". No n8n, o `Registrar falha` separa os dois 409 (obra com contrato × número repetido) e manda para revisão com "nossa equipe vai verificar se é um aditivo ou um reenvio". O `Registrar falha` também passou a ler o corpo da resposta da rota como JSON (antes, com aspas escapadas, a mensagem saía crua) — testado offline **4/4**.

Pendente: a mesma regra na tela ("Novo contrato", "Gerar contrato de proposta"). Não foi feita — a pergunta ao Breno ficou ambígua ("regra do contato") e ela mexe em código e testes da sprint 8, e o banco tem dados de teste que já a violam.

7.2 Conferência (decisão 29)

- **Migration `20260925130000_documentos_conferencia.sql`, aplicada em gc-dev**: coluna `conferencia` (`pendente`, `aceita`, `recusada`), com índice parcial das pendentes.
- **As rotas** gravam `conferencia: 'pendente'` ao criar.
- **Actions** `aceitarDocumento` (admin e comercial) e `recusarDocumento` (só admin): o Não aceitar exige motivo, só desfaz proposta ainda em rascunho ou contrato ainda ativo, solta a referência do documento antes de apagar (o `contrato_criado_id` não tem ON DELETE), apaga itens, o pai e os arquivos anexados, e **volta o documento se a exclusão falhar**.
- **`src/components/ConferenciaAutomacao.tsx`**: a faixa "criada pela automação — confira", no topo do detalhe do documento, da proposta e do contrato.
- **Listagem**: selo **"A conferir"** e filtro com o mesmo nome.

7.3 Números

VERIFICAÇÃO	RESULTADO
`documentos.test.ts`	**17/17**
`ingestao.test.ts` / `contatos.test.ts` / `obra-do-documento.test.ts`	**25/25**, **11/11**, **10/10**
`tsc --noEmit` / `next lint`	limpos
`Registrar falha` offline (os dois 409, 422 com e sem escape)	**4/4**
`n8n_validate_workflow`	0 erros

Plano estendido: o bloco de escrita da rota de contrato passou a criar **duas obras de teste próprias** (a regra nova recusaria contrato em obra do seed) e ganhou 7 passos (segundo contrato na obra → 409, a conferir, Aceitar, aceitar de novo recusado, comercial não desfaz, Não aceitar apaga tudo e devolve à revisão, número repetido noutra obra → 409); o **passo 31** do navegador cobre a faixa no detalhe da proposta. **Nada rodou.**

---

8. Números do fechamento da sprint 8 — 28/09

O fechamento da sprint 8 rodou `npm run validar` completo, feito pela sessão da sprint, e cobriu tudo o que este documento entregou até a seção 7: rota de contrato, revisão, conferência e os passos 30 e 31 do navegador. Números daquele run:

CAMADA	RESULTADO
estatico	limpo
unit	**291 casos**
build	**45 rotas**
runtime	**141/141 rotas**
dados	**36/36 checagens**
escrita	**390/390**
navegador	**154/154**

A limpeza do bloco da rota de contrato falhou numa primeira passada, por FK (`documentos_processamento_contrato_criado_id_fkey` e depois `contratos_obra_fk`). A ordem da limpeza foi corrigida (documentos → contratos → proposta → obras). Também entrou uma rede de segurança que desvincula `contrato_criado_id` e apaga contrato perdido nas obras de teste antes de apagar as obras.

As seções 1 a 7 ficam, portanto, **validadas**, com duas exceções que continuam valendo:

- a conferência visual pelo celular;
- a publicação das rotas, que depende do merge.

---

9. Log da automação: etapa e erro descritivo (decisão 30) — 28/09

Implementado, validação pendente para o fechamento da próxima rodada (ou quando o Breno pedir).

O que entrou

- **Banco (aplicado em gc-dev):**
  - `20260928110000_automacao_eventos.sql`: colunas de etapa no documento; tabela `automacao_eventos` com RLS de leitura e sem policy de insert; trigger que grava um evento a cada mudança de etapa, status ou conferência (nível erro, aviso ou info) e trigger que carimba `etapa_em`.
  - `20260928120000_automacao_eventos_ordem.sql`: `criado_em` passa a `clock_timestamp()`, porque dois eventos na mesma transação saíam fora de ordem.
  - Types regenerados.
- **n8n:**
  - `Processar Documento` (39 nós) grava a etapa em cada passo: LEITURA ao criar o registro; LEITURA_RESERVA, com o motivo, quando o Gemini falha e o Groq assume; GRAVANDO antes

da rota; CONCLUIDO ou REVISAO/ERRO no fim. Também grava CONCLUIDO com aviso quando o anexo falha, e um evento quando nem o registro inicial consegue ser criado.
  - `Notificar` grava todo erro de workflow em `automacao_eventos` (`origem = 'n8n'`, com workflow, nó e id da execução).
  - Os dois publicados com 0 erros de validação; teste offline dos nós novos **8/8**.
- **gc-sistema:**
  - `src/lib/automacao.ts`: etapas, rótulos, `descreverErro` e `situacaoDaAutomacao`.
  - `/documentos/[id]`: painel **Andamento da automação**, com a etapa atual, a linha do caminho e a linha do tempo dos eventos; erro e aviso vêm explicados, e a mensagem técnica fica recolhida.
  - `/documentos`: faixa vermelha **"A automação está parada: ..."** e link para o log; na listagem, a etapa aparece embaixo do status dos pendentes.
  - `/documentos/log` (rota nova): eventos paginados, filtro por nível na URL e link para o documento.
  - Envio e reprocessamento pela tela voltam a etapa a NA_FILA. Se o webhook do n8n falhar, o documento vai para ERRO com o corpo da resposta, e é por isso que "Execution limit reached" chega à tela traduzido.

Números

VERIFICAÇÃO	RESULTADO
`node --test src/lib/automacao.test.ts`	**9/9**, rodado ao criar o helper, com a mensagem real do limite do n8n
`tsc --noEmit`	limpo
`next lint`	sem avisos
Trigger em gc-dev, conferido à mão	LEITURA → LEITURA_RESERVA (aviso, cota do Gemini) → ERRO (erro), em ordem

Plano estendido, nada rodou

- `validacao-rotas.json`: 4 rotas.
  - `/documentos/log` como admin e como visualizador.
  - `?nivel=erro`.
  - Financeiro recebe 307.
  - `/documentos` também espera o link do log.
- `rotas-esperadas.txt`: 45 → 46, com `/documentos/log` inserido à mão na posição do `sort`.
- `validar-dados.mjs`: 3 checagens (andamento em ordem, eventos recentes, log filtrado). A listagem passou a selecionar `conferencia, etapa` e a aceitar `DESCARTADO`.
- `validar-escrita.mjs`: 2 passos.

    - O trigger grava recebimento e descarte, em ordem.
    - O usuário não insere evento direto.

- `validar-navegador.mjs`: **passo 34**, com documento de teste parado pelo limite do n8n. Confere o detalhe com etapa e erro traduzido, a faixa em `/documentos` e o log filtrado, e deixa screenshots `34-*.png`.

Pendências nominais

1. **`npm run validar` não rodou** para esta seção.
2. **Nenhum evento real de n8n exercitado ponta a ponta:** o n8n está no limite de execuções. O caminho do `Notificar` foi testado só offline.
3. **Documentos anteriores a 28/09 não têm andamento.** O painel diz isso em vez de ficar vazio.
4. **Visualizador não vê os erros do n8n que não têm empresa** (`empresa_id` nulo). A RLS os mostra só a admin e comercial, de propósito.
5. Conferência visual no celular: **não feita**.

9.1 Primeiro teste real — 01/10

- **Token:** o `INGESTAO_TOKEN` da Vercel não conferia com o da Credential do n8n, e a rota devolvia 401. O Breno trocou o token nos três lugares (Vercel, n8n e `.env.local`); depois disso a rota aceitou.
- **Obra não identificada:** o documento `53c5f7c8` (EB-25-08-0044 · ESTAÇÃO FASHION - FACHADA) foi para revisão. É o comportamento esperado pela decisão 27: a obra não existe no gc-dev, que só tem três obras de teste.
- **Bug corrigido no n8n — etapa sobrescrita.**
    - **Sintoma:** o documento `11e492b9` ficou com a etapa "lendo pela reserva" estando em revisão.
    - **Causa:** no `executionOrder: v1`, o ramo de cima do canvas roda primeiro. O "Etapa - Reserva Groq" estava embaixo, por isso só rodava depois do caminho inteiro do Groq e regravava a etapa.
    - **Correção:** o nó subiu no canvas. Os PATCH de etapa ganharam trava na URL: `etapa=eq.LEITURA` no da reserva e `etapa=in.(LEITURA,LEITURA_RESERVA)` no de gravação. Assim, nunca sobrescrevem uma etapa posterior, qualquer que seja a ordem de execução.
    - Publicado, com 0 erros no `n8n_validate_workflow`.

10. O PDF vira anexo pela rota, não pelo n8n — 01/10

Implementado, validação pendente (`npm run validar` não rodou).

O que mudou

- **`src/lib/anexo-do-documento.ts`** (novo):
    - `anexarPdfDoDocumento` copia o PDF do bucket `documentos-processamento` para `anexos`, em `{empresa}/{propostas|contratos}/{id}/{ts}_{nome}` (`buildStoragePath`, o formato que a RLS do Storage e a aba Anexos esperam).

  - Acrescenta a entrada no jsonb `anexos`, com o profile de serviço como autor. Não duplica numa reexecução.
  - Se não conseguir registrar a entrada, apaga o arquivo para ele não ficar órfão.
  - Tem duas funções puras com teste: `nomeOriginalDoArquivo` e `juntarAnexo`.

- **Rotas `/api/ingestao/proposta` e `/contrato`:**
  - Anexam o PDF logo depois de vincular o documento.
  - A resposta ganhou `anexo` (o caminho do anexo, ou null) e `anexoErro`.
  - Falha no anexo **não desfaz** o registro. Volta como `anexoErro`, que o n8n coloca no aviso ao grupo e no `etapa_detalhe` (o painel de andamento mostra).

- **n8n `Processar Documento` (publicado, 40 nós, 0 erros de validação):**
  - Novo IF **"Anexo pela rota?"** depois de "Documento aprovado". Se a resposta da rota tem o campo `anexo`, a rota já anexou, e o fluxo vai direto para os avisos. Se não tem, é a rota antiga, e o fluxo segue pelos 4 nós de anexo de antes.
  - Funciona com a rota antiga e com a nova, então a ordem entre o deploy da Vercel e o n8n não importa: nunca anexa duas vezes nem deixa de anexar.
  - "Documento aprovado" grava `PDF não anexado: …` em `etapa_detalhe` quando a rota devolve `anexoErro`.
  - "Montar avisos" lê o resultado do anexo da resposta da rota.

Números

VERIFICAÇÃO	RESULTADO
`node --test src/lib/anexo-do-documento.test.ts`	**3/3**
`tsc --noEmit` / `next lint`	limpos
`n8n_validate_workflow`	0 erros, 40 nós

Plano estendido, nada rodou

`validar-escrita.mjs` ganhou **4 passos** no bloco da rota de proposta:

1. Documento cujo PDF não está no bucket: a proposta fica, com `anexo` null e `anexoErro` preenchido.
2. Com um PDF real no bucket: a rota cria a entrada no jsonb com o nome original, o autor de serviço e o caminho `{empresa}/propostas/{id}/`.
3. O arquivo existe em `anexos` com o mesmo tamanho.
4. Limpeza da proposta, do anexo e do PDF de origem.

Pendências nominais

1. **`npm run validar` não rodou.**

2. **Não publicado:** a rota nova precisa de commit e merge. Até lá, a Vercel roda a rota antiga, e o n8n anexa pelo caminho de antes.
3. **Depois do deploy e de um envio real com sucesso:** apagar do n8n os 5 nós do caminho antigo ("Preparar anexo", "Supabase - Subir anexo", "Supabase - Registrar anexo", "Anexo falhou", "Etapa - Anexo falhou") e o IF.
4. **Rota de contrato:** o anexo pela rota só é exercitado na de proposta; a de contrato usa o mesmo helper.

10.1 No ar — 02/10

Commit e merge feitos pelo Breno (`38f446a`). O deploy de Production na Vercel foi concluído. Teste direto na rota publicada, com um PDF de teste no bucket e a obra OBRA-2025-03:

- **201**, com `anexo` = `{empresa}/propostas/{id}/…_smoke-anexo.pdf` e `anexoErro` null.
- 1 entrada no jsonb `anexos` , com o nome original e 23 bytes; o arquivo está no bucket `anexos` com o mesmo tamanho.
- Limpeza feita: proposta, itens, anexo, documento e PDF de origem.

Ainda falta o envio real pelo bot, que é o que leva o n8n pelo IF "Anexo pela rota?".

10.2 Teste completo pelo bot — 07/10

O Breno cadastrou a obra **Residencial Vista Verde - Bloco B** (`BRA-2026-005`) e mandou pelo bot o par de teste em `documentos-fonte/teste-vinculo-1n/` .

DOCUMENTO	LEITURA	RESULTADO
`6fac05a3` , proposta PROP-2026-0117	Gemini	proposta de R$ 248.500,00 na obra Vista Verde, achada pelo nome, com 4 itens e o PDF em Anexos; etapa CONCLUÍDO, conferência pendente
`c5e61c09` , contrato CT-2026-0042	Groq (o Gemini falhou: aviso no andamento)	contrato de R$ 248.500,00 na mesma obra, **ligado à proposta PROP-2026-0117**, com 4 itens e o PDF em Anexos; etapa CONCLUÍDO

Andamento gravado: LEITURA → (LEITURA_RESERVA) → GRAVANDO → CONCLUIDO. Nenhum erro do n8n.

Depois do teste:

- **n8n:** os 6 nós do caminho antigo de anexo saíram ("Anexo pela rota?", "Preparar anexo", "Supabase - Subir anexo", "Supabase - Registrar anexo", "Anexo falhou", "Etapa - Anexo falhou"). "Documento aprovado" vai direto para "Montar avisos", que lê `anexoErro` da rota. Publicado, com 33 nós e 0 erros.
- **Nome do anexo:** pelo bot, o anexo saía com o update_id do Telegram na frente (`174281296-PROPOSTA_…`). `nomeOriginalDoArquivo` passou a tirar esse prefixo também. O teste do helper

continua passando (3/3, com dois casos novos), e `tsc` e `lint` estão limpos. A correção vale depois do próximo commit e merge; os dois anexos já gravados ficam com o nome antigo.

Pendências:

- O `npm run validar` não rodou.
- Conferência visual das telas (Anexos e andamento) não feita por mim.

11. O bot pergunta a obra (decisão 31) — 07/10

Implementado, validação pendente (`npm run validar` não rodou).

O que entrou

- **Banco (aplicado em gc-dev):** migration `20261008100000_documentos_opcoes_obra.sql`, com a coluna `opcoes_obra jsonb` e um índice parcial por chat. Types regenerados.
- **`src/lib/obra-do-documento.ts`:** `opcoesParaPergunta`, `textoDaPergunta`, `interpretarRespostaDeObra` e `lerOpcoesGravadas`.
- **Rotas de proposta e contrato:**
  - No 422 de obra não identificada, gravam as opções no documento e devolvem `pergunta`.
  - As obras candidatas vêm da mais recente para a mais antiga.
  - No sucesso, limpam `opcoes_obra`.
- **Rota nova `POST /api/ingestao/resposta-obra`:** autentica por token, como as outras de ingestão, e devolve uma de quatro ações: `sem_pergunta`, `repetir`, `nenhuma` ou `reprocessar` (com a `entrada` para o n8n).
- **n8n `Processar Documento`** (publicado, 37 nós, 0 erros):
  - **"Triar mensagem":** texto de um contato cadastrado vira `acao: 'resposta'`, em vez do aviso fixo.
  - **Nós novos:** "Resposta de texto?" → "Rota - Resposta da obra" → "Tratar resposta" → "Reprocessar?". Com a obra escolhida, o fluxo volta ao "Normalizar entrada" com a obra fixada.
  - **"Registrar falha"** guarda a `pergunta` do 422.
  - **"Montar avisos"** manda a pergunta a quem enviou, com o aviso `[PERGUNTA DE OBRA]` ao grupo.
  - **Antes do deploy:** a rota de resposta ainda dá 404, o "Tratar resposta" cai no aviso de sempre, e a pergunta não é feita porque a rota antiga não devolve `pergunta`. Nada quebra.

Números

VERIFICAÇÃO	RESULTADO
`node --test src/lib/obra-do-documento.test.ts`	**14/14** (10 antigos + 4 novos)
`tsc --noEmit` / `next lint`	limpos
`n8n_validate_workflow`	0 erros, 37 nós

Plano estendido, nada rodou

- **`validar-escrita.mjs`, 9 passos no bloco da rota de contrato:**
  1. 422 com a pergunta e as opções gravadas.
  2. `resposta-obra` sem token → 401.
  3. Outro chat → `sem_pergunta`.
  4. Texto → `repetir`.
  5. Número fora da lista → `repetir`.
  6. "1" → `reprocessar`, com o documento PENDENTE, a obra da opção 1 e a pergunta fechada.
  7. Repetir o "1" → `sem_pergunta`.
  8. "0" → `nenhuma`, em revisão, com o motivo.
  9. Limpeza pelos documentos do bloco.
- **`rotas-esperadas.txt`**: + `/api/ingestao/resposta-obra`.

Pendências nominais

1. **`npm run validar` não rodou.**
2. **Não publicado:** precisa de commit e merge.
3. **Teste pelo bot depois do deploy:** mandar um PDF de obra não cadastrada, responder um número e depois "0" noutro envio.
4. **A releitura gasta mais uma chamada de IA:** o documento volta para a fila e o PDF é lido de novo. Reaproveitar a leitura salva fica como melhoria, se o custo pesar.

12. O bot avisa quem enviou e alerta o grupo quando a automação para — 08/10

Implementado, validação pendente (`npm run validar` não rodou). Depende de uma variável nova, `TELEGRAM_BOT_TOKEN`.

Decisão de desenho

O gc-sistema passa a mandar estas mensagens **direto pela API do Telegram**, sem o n8n.

- **Por quê:** não gasta execução do n8n e é o único jeito de avisar o "limite de execuções do n8n". Com o n8n no limite, nada nele roda, nem o próprio aviso.
- **Token:** fica só na variável de ambiente. Nunca em código, documento ou log.

O que entrou

- **`src/lib/telegram.ts` (novo).**
  - Partes puras e testadas:
    - `avisoDeResolucao`: textos de aceito, não aceito, vinculado e descartado, no gênero certo para proposta e contrato.
    - `alertaDeAutomacaoParada`: título, explicação e "o que fazer", de `descreverErro`.

    - `deveAlertar`: um alerta por causa a cada 6 h.
    - `chatIdValido`.
  - `enviarTelegram`: um `sendMessage` com timeout de 5 s. Nunca lança erro. Sem token, não envia.

- **Ações de `/documentos` (Aceitar, Não aceitar, Vincular e Descartar).** Depois de feitas, avisam quem mandou o PDF.
  - Só se o documento veio pelo bot e o chat é um chat real do Telegram.
  - Falha no envio não desfaz a ação: volta como aviso amarelo na tela ("Feito, mas...").

- **Alerta ao grupo de admin.** Quando o envio ou o reprocessamento pela tela não consegue acionar o n8n por um erro que para tudo (limite de execuções, token ou configuração), o grupo recebe um `[AUTOMAÇÃO PARADA]`.
  - Cada alerta fica registrado em `automacao_eventos` (etapa `ALERTA`, código no `detalhe`). É isso que segura a repetição dentro da janela, e o log da automação mostra o registro como "Alerta ao grupo do Telegram".
  - O grupo vem de `TELEGRAM_ADMIN_CHAT_ID` ou, sem ela, do mesmo grupo do n8n.

Números

VERIFICAÇÃO	RESULTADO
`node --test` telegram, automacao e documentos	**30/30** (4 novos em `telegram.test.ts`)
`tsc --noEmit` / `next lint`	limpos

Plano estendido, nada rodou

`validar-escrita.mjs` ganhou 1 passo: descartar um documento de chat fictício não tenta avisar pelo Telegram (a ação volta sem aviso).

O envio real não é exercitado pelo plano: o chat dos documentos de teste é fictício de propósito, para a validação não mandar mensagem a ninguém.

Pendências nominais

1. **Configurar `TELEGRAM_BOT_TOKEN`:** o token do bot de produção, na Vercel (Production, com Redeploy) e no `.env.local`. Sem ela, nada é enviado.
2. **Commit e merge.**
3. **Teste real:** aceitar ou descartar pela tela um documento enviado pelo bot e conferir a mensagem no Telegram. O alerta de automação parada só aparece num erro de verdade; para ver antes, dá para testar com o token do n8n trocado de propósito.
4. **Limite que continua:** PDF enviado pelo bot enquanto o n8n está no limite não chega a lugar nenhum, porque o Telegram entrega ao n8n e ele não roda. O alerta só sai quando alguém usa a tela. Para cobrir isso, seria preciso uma verificação periódica do n8n, que fica para depois.

5. `npm run validar` não rodou.